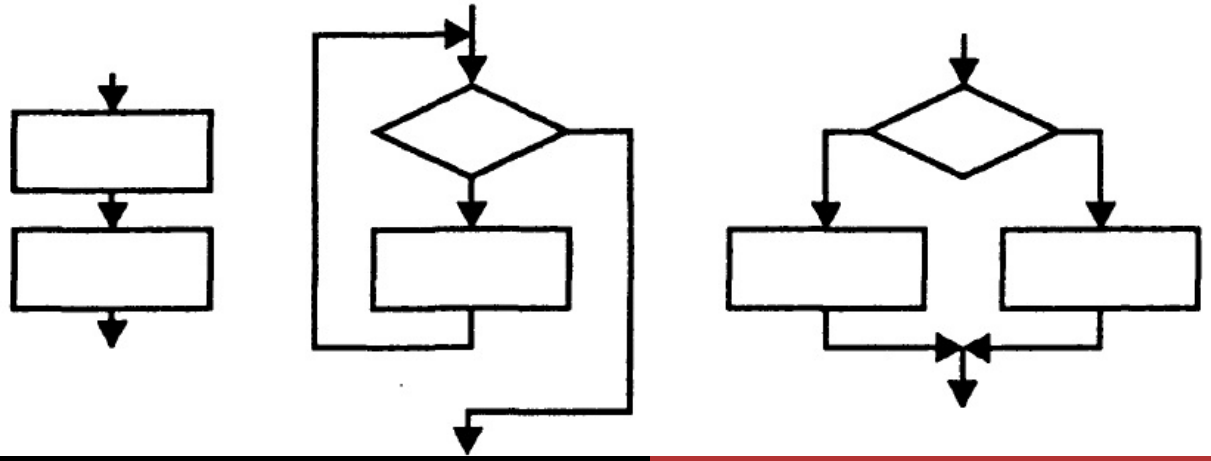
Лекция III

25 сентября 2020

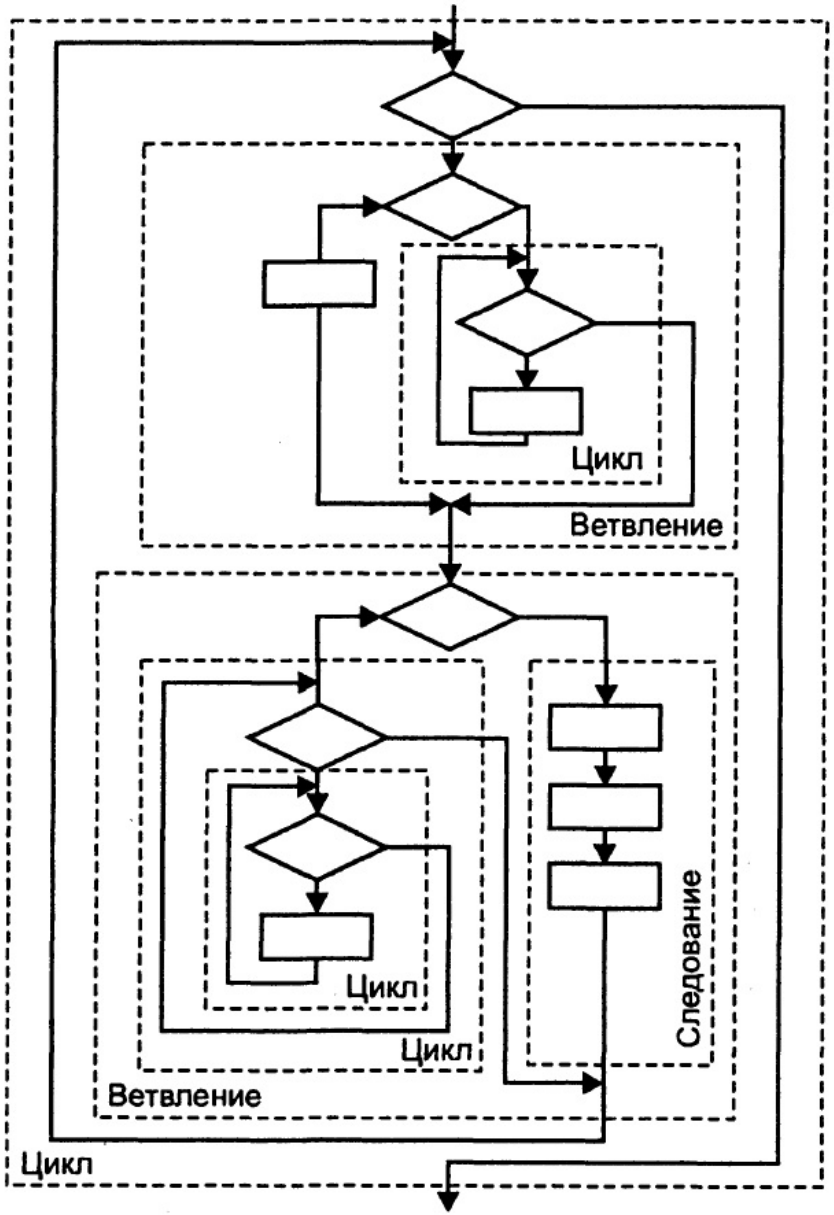
Идея структурного программирования

Любую программу можно составить только из структур трех типов: следования, ветвления и цикла (это базовые конструкции).

1. **Следованием** называется конструкция, представляющая собой последовательное выполнение двух или более операторов (простых или составных).
2. **Ветвление** задает выполнение либо одного, либо другого оператора в зависимости от выполнения какого-либо условия.
3. **Цикл** задает многократное выполнение оператора.



Идея структурного программирования



Условный переход: общий синтаксис

```
if ( <логическое выражение> ) {
  <набор инструкций>
} [else {
  <набор инструкций>
}]
```

```
1 int max_score = 3, min_score = 5;
2
3 if ( max_score < min_score )
4     printf("Как-то странно\n");
5 printf("эта инструкция не относится к ↩
    оператору ветвления");
```

Условный переход: пример

```
1  int max_score = 3, min_score = 5;
2  // Так гораздо понятнее:
3  if ( max_score < min_score ) {
4      printf("Как-то странно\n");
5  }
6  printf("Эта инструкция не относится к ↩
       оператору ветвления");
7
8  int current_score = 4;
9  if ( (current_score >= min_score) && (↩
       current_score <= max_score) ) {
10     printf("Всё ок");
11 } else {
12     printf("Какая-то аномалия");
13 }
```

Условный переход: ещё один пример

```
1  int current_score = 7;
2
3  if ( current_score == 4 ) {
4      current_score += 4;
5  } else if ( current_score == 5 ) {
6      current_score += 1;
7  } else if ( current_score == 6 ) {
8      current_score -= 10;
9  } else {
10     current_score = 0;
11 }
```

Тернарный оператор **?:**

```
<логическое выражение>
    <?> <выражение, если истинно>
    <:> <выражение, когда ложно>;
```

```
1 int max_score = 13, min_score = 5;
2
3 int result;
4 result = ( (max_score - min_score) < 5 ) ? ↩
    min_score * 2 : max_score / 2; )
```

Конструкция **switch**

```
switch (<выражение>) {
    case <значение_1>:
        [инструкции]
        <break>;
    case <значение_2>:
        [инструкции]
        <break>;
    ...
    case <значение_T>:
        [инструкции]
        <break>;
    [default:
        [инструкции]
    ]
}
```

2. Управление ходом выполнения программы

```
1  int rate, score;
2  // Вычисляем rate
3
4  switch ( rate ) {
5    case 2:
6      score = 1; break;
7    case 5:
8      score = 2;
9      break;
10   case 8:
11     printf("Выпала восьмёрка!\n");
12     score = 3;
13     break;
14   default:
15     score = 5;
16 }
```

2. Управление ходом выполнения программы

- Фактически **switch** производит сопоставления результата выражения с значениями в "ветках"**case**
- Его недостатком является то, что результат выражения должен быть приводим к числу

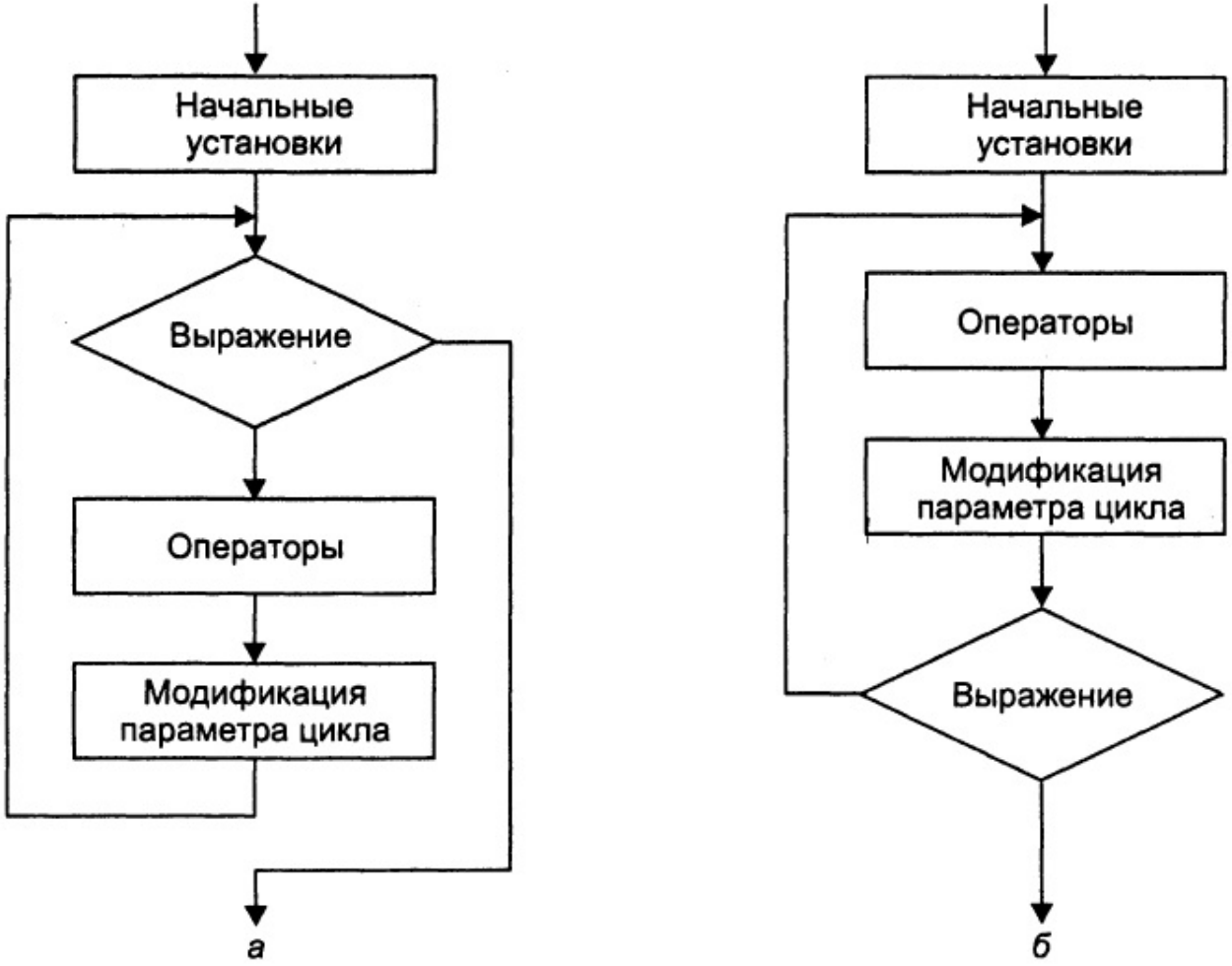
Цикл

Под **циклом** в программировании понимается обособленный набор повторяющихся N раз инструкций. Причём, $N \geq 0$

Дополнительные определения

- **Тело цикла** - набор повторяющихся инструкций
- **Итерация** (или проход цикла) - однократное выполнение всех инструкций из тела цикла

Циклы с условием



Структурные схемы операторов цикла: *а* — цикл с предусловием; *б* — цикл с постусловием

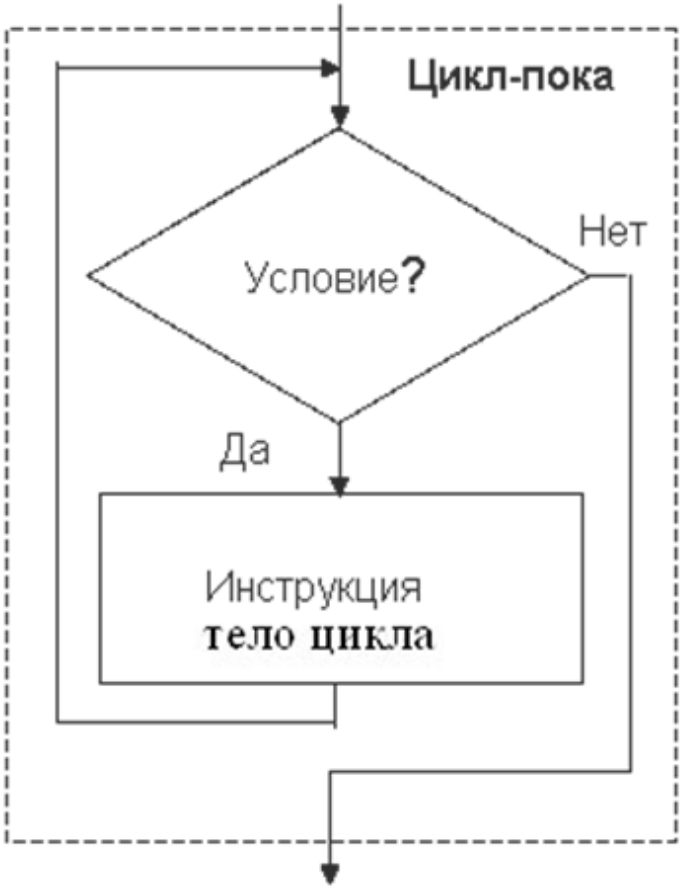
Циклы с условием

Циклы **while** и **do ... while**

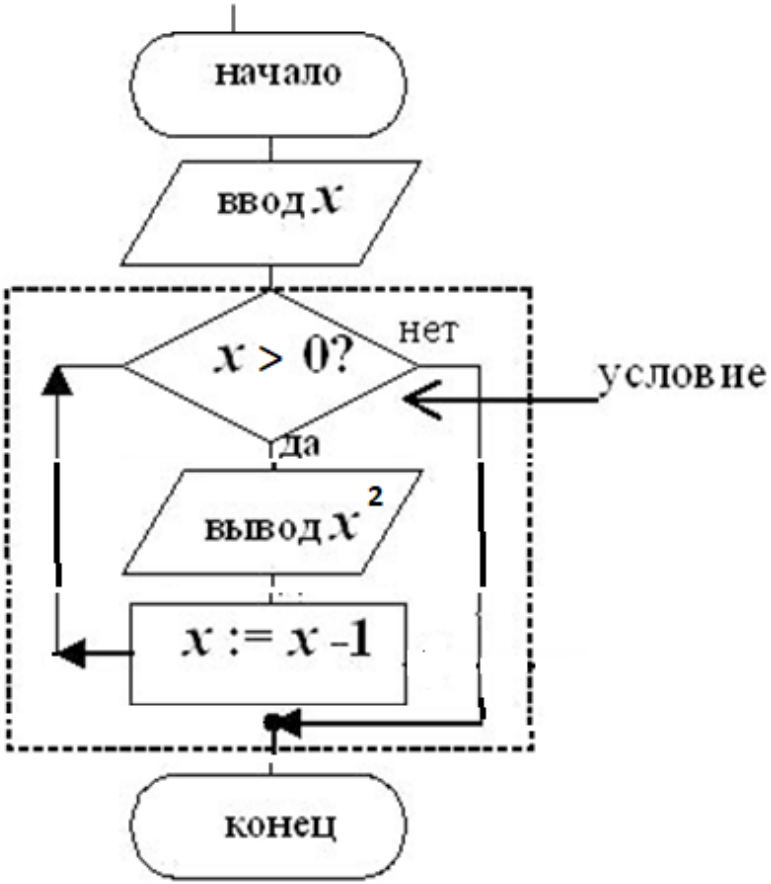
```
while (<логическое выражение>) {
   [инструкции];
}

do {
   [инструкции];
} while (<логическое выражение>);
```

Цикл с предусловием



цикл с предусловием

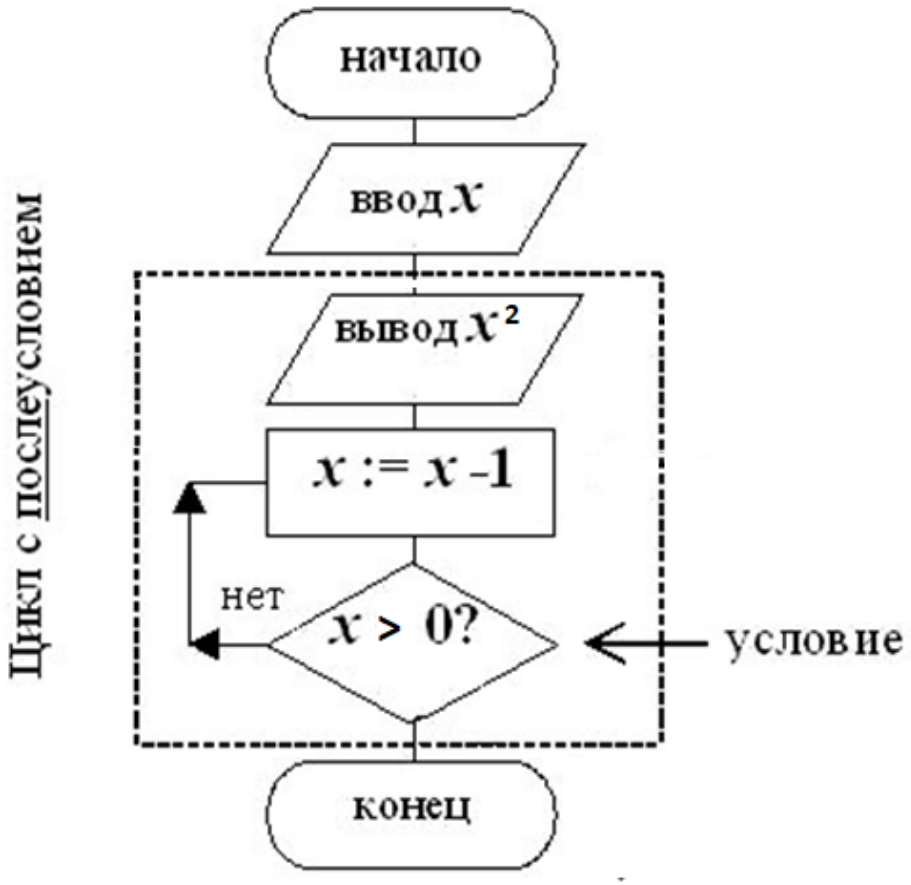


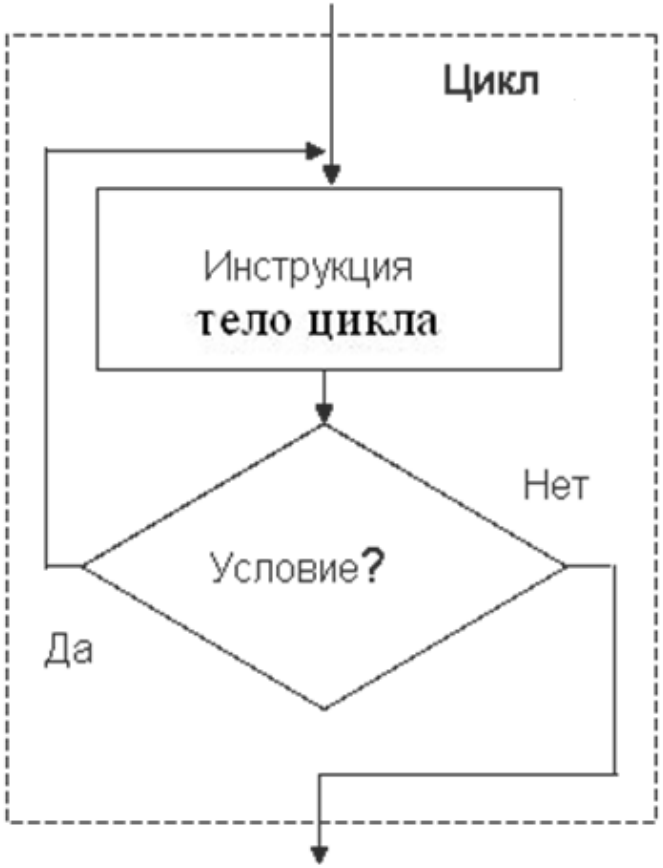
Цикл с предусловием

Цикл **while**

```
1 int counter = 10;
2
3 while ( counter > 0 ) {
4   printf("%d\n", counter * counter);
5   counter--;
6 }
```

Цикл с постусловием





Цикл с постусловием

Цикл **do ... while**

```
1 int counter = 10;
2
3 do {
4   printf("%d\n", counter * counter);
5   counter--;
6 } while ( counter > 0 );
```

Найти количество цифр числа и их сумму

```
1 #include <stdio.h>
2 int main()
3 { int N; // число, с которым работаем
4 int count=0, cifra, summa=0;
5 printf("\nВведите число N: ");
6 scanf ( "%d", &N ); // ввод N с клавиатуры
7 while ( N > 0 ) // заголовок цикла
8 { // начало цикла
9 cifra=N%10; // определяем последнюю цифру
10 summa+=cifra; // добавляем в сумму
11 N /= 10; // отсекаем последнюю цифру
12 count ++; // увеличиваем счетчик цифр
13 } // конец цикла
14 printf("В числе %d цифр их сумма %d\n", ↩
     count, summa);
15 return 0;}
```

Проверка на ввод натурального числа.

```
1  #include <stdio.h>
2  int main()
3  {
4    int N;
5    do { // начало цикла
6      printf("\nВведите натуральное число:");
7      scanf( "%d", &N );
8    }
9    while ( N <= 0 ); // условие цикла
10  printf ( "OK");
11  return 0;
12 }
```

Бесконечный цикл на примере **while**

```
1 while( 1 ) {
2   printf("*");
3 }
```

Цикл с известным числом шагов(с параметром)

Цикл **for**

```
for (   [инициализация];
        [условие выхода];
        [итеративное изменение]
    ) {
  [инструкции];
}
```

1. **Инициализация**: инструкции, которые выполнятся до начала работы цикла. Как правило, задание различных счётчиков, используемых внутри цикла.
2. **Условие выхода**: логическое выражение, проверяемое **до первой** и после каждой следующей итерации.
3. **Итеративное изменение**: задание выражений (изменение счётчика цикла, как пример), которые выполняются **после каждой итерации**.

Цикл с известным числом шагов

Цикл **for**

```
1
2 // Вывод значений квадратного корня для ↩
    чисел от 0 до 9
3 for (int i = 0; i < 10; i++) {
4   printf("%lf\n", sqrt(i));
5 }
6 for (i=0, x=1.; i<10; i+= 2, x*= 0.1)
7     { printf("%lf\n", x); }
8
9 // Бесконечный цикл
10 for (;;) {
11   // что-то полезное
12 }
```

Управление циклами continue и break

```
1 #include <stdio.h>
2 int main()
3 {int A, B;
4 while ( 1 ) // бесконечный цикл
5 { printf("\nВведите делимое и делитель:");
6   scanf ( "%d%d", &A, &B );
7   if ((A==0)&&(B==0)) break; //Выход из ↩
      цикла
8   if ( B == 0 ) {
9     printf ( "Деление на ноль" );
10    continue;//переход к новому шагу цикла
11  }
12  printf("Частное %d,остаток %d", A/B, A%B);
13 }
14 return 0;}
```

Часто встречающиеся ошибки при программировании циклов

использование в теле цикла неинициализированных переменных и неверная запись условия выхода из цикла. Чтобы избежать ошибок, рекомендуется:

1. проверить, всем ли переменным, встречающимся в правой части операторов присваивания в теле цикла, присвоены до этого начальные значения;
   Например, если до выполнения цикла `while (a < 1.0) a*=2.0;` переменная `a<=0`, то произойдет «зацикливание».
2. проверить, изменяется ли в цикле хотя бы одна переменная, входящая в условие выхода из цикла;
3. предусмотреть аварийный выход из цикла по достижению некоторого количества итераций;
4. если в теле цикла требуется выполнить более одного оператора, нужно заключать их в фигурные скобки.

Операторы цикла взаимозаменяемы, но можно привести некоторые рекомендации по выбору наилучшего в каждом конкретном случае.

1. Оператор **do while** обычно используют, когда цикл требуется обязательно выполнить хотя бы раз (например, если в цикле производится ввод данных).
2. Оператором **while** удобнее пользоваться в случаях, когда число итераций заранее не известно, очевидных параметров цикла нет или модификацию параметров удобнее записывать не в конце тела цикла.
3. Оператор **for** предпочтительнее в большинстве остальных случаев (однозначно — для организации циклов со счетчиками).

Вычисление сумм последовательностей с заданным числом элементов

Найти сумму первых 20 элементов последовательности
$s = \frac{1}{2} - \frac{2}{4} + \frac{3}{8} - \frac{4}{16} + \dots$
Чтобы решить эту задачу, надо определить закономерность в изменении элементов

1. каждый элемент представляет собой дробь;
2. числитель дроби при переходе к следующему элементу возрастает на единицу;
3. знаменатель дроби с каждым шагом увеличивается в 2 раза;
4. знаки перед дробями чередуются (плюс, минус и т.д.).

Вычисление сумм последовательностей с заданным числом элементов

```
1 #include <stdio.h>
2 int main()
3 { float Sum = 0, znak = 1, chisl = 1,
4         znam = 2; // начальные значения
5 for (int i = 1; i <= 20; i ++ )
6 {    Sum = Sum + znak*chisl/znam; // добавить↩
      элемент к сумме
7      znak = - znak;
8      chisl ++;
9      znam = znam * 2;
10 }
11 printf("Сумма S = %f", Sum);
12 return 0;}
```

приближенное вычисление суммы

Пример с приближенным вычислением суммы очень важен т.к многие встречающиеся в математике и физике функции могут быть представлены бесконечными суммами:

$$y = \cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \ldots (-1)^n \frac{x^{2n}}{n!} + \ldots$$

$$y = \sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots (-1)^n \frac{x^{2n-1}}{(2n-1)!} + \ldots$$

Величина $n!$ называется «эн факториал», ее значение равно $1 \cdot 2 \cdot 3 \cdots n$. Значение $n!$ с увеличением n быстро растет. В частности, для любого x можно найти n такое, что $n! > x^n$.

приближенное вычисление суммы

- Эта задача имеет решение только тогда, когда элементы последовательности убывают по модулю и стремятся к нулю.
- нужное приближение получено, если вычислена сумма нескольких первых слагаемых и очередное слагаемое оказалось по модулю меньше, чем данное малое положительное число ε, – это и все последующие слагаемые уже не надо учитывать.
- Поскольку мы не знаем, сколько элементов войдет в сумму, надо использовать цикл while (или do-while).

приближенное вычисление суммы

$s = \sum_{i=1}^{\infty} \frac{x^{2i-1}}{(2i-1)!}$.

```
1 #include <stdio.h>
2 #include <math.h>
3 int main()
4 { float Sum = 0, eps=0.0001, x, slag;
5  int k=1;
6 scanf("%f",&x);
7 slag=x; // начальные значения
8 while (fabs(slag)>eps )
9 {    Sum = Sum + slag;
10     k+=2;
11     slag*=x*x/(k*(k-1));
12 }
13 printf("Сумма S = %f", Sum);
14 return 0;}
```

Вложенные циклы

1. При организации вложенных циклов все правила, присущие простому циклу, должны соблюдаться.
2. Имена параметров для циклов, вложенных один в другой, должны быть различными.
3. Внутренний цикл должен полностью входить в тело внешнего цикла. Пересечение циклов недопустимо.
4. Для лучшего восприятия программы рекомендуется вложенные структуры записывать на 1 строку ниже и на 1 – 2 позиции правее.
5. Глубина вложения - количество внешних циклов, охватывающих данный цикл. Если внешний цикл выполняется N_0 раз, цикл с глубиной вложения 1 выполняется N_1 раз, то тело цикла с глубиной вложения m будет выполняться $N_0 * N_1 * .. * N_m$ раз.

Вложенные циклы

```
1  #include <stdio.h>
2  int main()
3  { float Sum1=0, Sum2=0, eps=0.001;
4    int k=1, i;
5    do
6    { Sum2=1;
7      for(i=1;i<=k;i++) Sum2+=i;
8      Sum1=Sum1+1/Sum2;
9      k++;
10   } while((1/Sum2)>eps);
11 printf("Сумма S = %f", Sum1);
12 return 0;}
```

Вложенные циклы

Не всегда использование вложенных циклов оправданно. Иногда можно выразить переменную на данном шаге итерации, через нее же на предыдущем шаге. Здесь факториал стоит не вычислять с 1, а использовать $n! = n * (n-1)!$.

```
1 {с вложением циклов}{без вложения циклов}
2 Sum=0; k=1;                Sum=0; k=1; fact=1;
3 do                         do
4 { fact=1;                   { fact*=k;
5   for (i=1;i<=k;i++)          Sum1+=1/fact;
6     fact*=i;                  k++;
7   Sum1+=1/fact;             } while ((1/sum2)>eps)↩
      ;
8   k++;
9 } while ((1/sum2)>eps);
```

Управляющие конструкции закончились: кроме трёх циклов, условного перехода да конструкции **switch** больше нечем влиять на логику программы (99% правда)

Переходим к рассмотрению функций в C

3. Функции. Определение

Функция - обособленный набор инструкций, пригодный для повторного использования, который может принимать произвольное количество значений и возращать значение. Определение функции в C

```
[...] <тип_возращаемого_значения>
      <имя_функции> ( <список_аргументов> )
{
  [инструкции;]
  return <значение>;
}
```

, где **список_аргументов** - это набор пар *тип данных* и *символьное обозначение* каждого параметра, перечисляемых через запятую. Троеточия - специальные указания компилятору, возможно рассмотрятся на практике.

3. Функции. Определение

- Каждая функция должна быть уникальна
- Уникальность функции в языках программирования определяется её **сигнатурой** - набором отличительных признаков
- В C сигнатура функции определяется **её именем** (строго говоря - идентификатором)
- Определение функции внутри другой функции стандартом языка запрещено (но некоторые компиляторы позволяют это делать)

$$n! = 1 * 2 * 3 * 4 ... * n$$

```
1  long long factorial(unsigned n)
2  {
3    long long result = 1;
4
5    for (unsigned i = 2; i <= n; ++i) {
6      result *= i;
7    }
8
9    return result;
10 }
11
12 ...
13 long long result = factorial(8);
14 printf("Факториал 8 равен: %lld", result);
```

3. Функции. Множественный return

Число операторов возврата **return** - неограничено.

```
1  long long factorial(unsigned n)
2  {
3    if ( n < 2) {
4      return 1;
5    }
6
7    long long result = 1;
8
9    for (unsigned i = 2; i <= n; ++i) {
10     result *= i;
11   }
12
13   return result;
14 }
```

number^n, n – целое

```
1  double degree_nth(double num, int n)
2  {
3    double result = 1;
4    int is_negative = n < 0;
5    unsigned degree = is_negative ? -n : n;
6
7    while ( degree > 0 ) {
8      result *= num;
9      --degree;
10   }
11
12   return is_negative ?
13           (1 / result) : result;
14 }
```

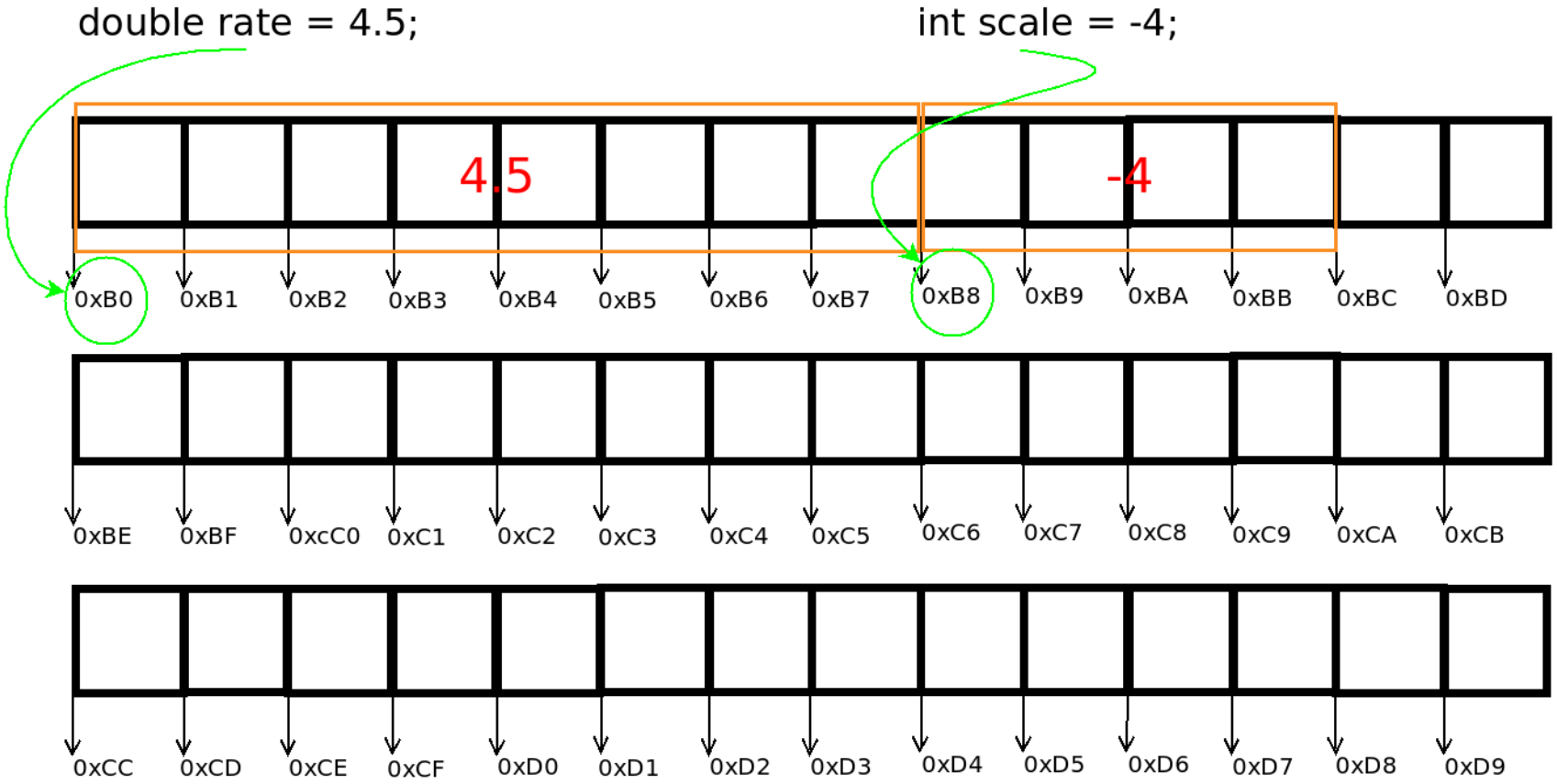
```
1 double degree_nth(double num, int n)
2 { ... }
3
4 double rate = 4.5;
5 int scale = -4;
6
7 double rate_in_scale = degree_nth( rate, ↩
    scale );
```

Прежде, чем идти дальше

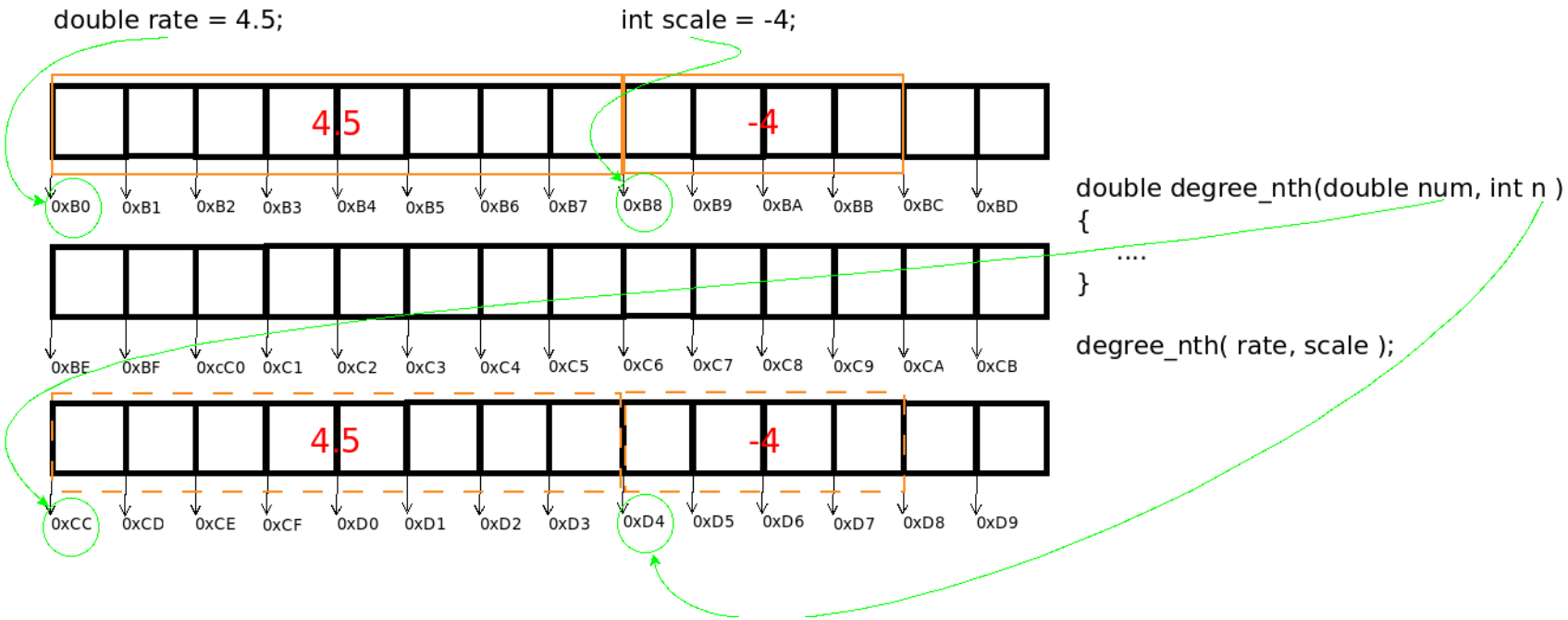
Адрес переменной

Каждая переменная любого типа связана с блоком в памяти, который состоит из какого-либо количества байт. Каждый байт в модели памяти языка C пронумерован - ему присвоен порядковый номер: 0, 1, 2, 3, ... Номер первого байта блока называется **адресом переменной**. Адреса в литературе приводят в *шестнадцатиричном* виде.

3. Функции. Передача аргументов по значению



3. Функции. Передача аргументов по значению



3. Функции. Специальный тип void

Функция может не возращать никакого значения. Для этого в качестве возращаемого типа используется специальный тип - **void**. Вызов оператора **return** не обязателен.

```
1 void print_pair(double first, double second)
2 {
3   printf("<< %lf, %lf >>", first, second);
4
5   // В тоже время, нельзя создать переменную↩
        типа void
6   // void v_var; // Ошибка компиляции!
7 }
```

3. Функции. Специальный тип void

Однако, оператор **return** может присутствовать для немедленного выхода из подобной функции (тип возращаемого значения - **void**).

```
1  void declarate_party(size_t alcohol_litres)
2  {
3    if ( alcohol_litres == 0 ) {
4      return;
5    }
6
7    if ( alcohol_litres < 10 ) {
8      printf("Party good: :)\n");
9      return;
10   }
11
12   printf("Party very good: :D!\n");
13 }
```

3. Функции. Объявление и определение

Аналогично переменным, для функций существуют понятия **объявления** и **определения**. Определение - это все примеры выше, а под **объявлением** понимается указание *типа возращаемого значения, названия функции и перечисление типов всех аргументов*. Причём, давать имена аргументам - не обязательно

```
1  #include <stdio.h>
2
3  double my_random(unsigned); // Объявление!
4
5  int main()
6  {
7    printf("Случайное число: %lf\n", my_random(567)↩
       );
8  }
9
10 double my_random(unsigned seed) // Определение!
11 { return 4.5688 / seed; }
```

Прежде, чем идти дальше

Область видимости идентификатора

Место в файле с исходным кодом, где идентификатор доступен для операций (использование значения в случае переменной, вызов - в случае функции). В языке C различают две области - **глобальная** и **локальная**.

- Все функции имеют **глобальную** область видимости
- Переменная имеет **локальную** область видимости, если её объявление или определение находится внутри пары фигурных скобок ({})
- Иначе переменная получает **глобальную** область видимости
- Локальная переменная видна во всех вложенных областях

Прежде, чем идти дальше

Область видимости идентификатора

```
1  // Переменная с глобальной областью видимости
2  const double PI = 3.14159265358;
3
4  void do_something(double x)
5  {
6    // Локальная область видимости
7    double rate = 0.5;
8
9    if (x > 10.0) {
10     // Ещё одна. PI и rate тут доступны
11     double num = PI * x * rate;
12     printf("Результат: %lf", num);
13   }
14
15   // Доступа к num уже нет:
16   // num += 3.05;
17   rate *= 9.5;
18 }
```

Прежде, чем идти дальше

Время жизни переменной

- Переменные с локальной областью видимости автоматически удаляются по достижении конца области видимости
- Глобальные переменные существуют до тех пор, пока выполняется программа

Структура

- Структура — это объединение нескольких объектов, возможно, различного типа под одним именем, которое является типом структуры. В качестве объектов могут выступать переменные, массивы, указатели и другие структуры.
- Структуры позволяют трактовать группу связанных между собой объектов не как множество отдельных элементов, а как единое целое.
- Структура представляет собой сложный тип данных, составленный из простых типов.

Структуры

```
struct тип_структуры
{
  тип ИмяЭлемента1;
  тип ИмяЭлемента2;
  . . .
  тип ИмяЭлементаn;
};
```

Пример объявления структуры

```
1  struct date
2  {
3    int day;      // 4 байта
4    int month;    // 4 байта
5    int year;     // 4 байта
6  };
7  struct point
8  {
9    double x;
10   double y;
11   double z;
12 };
```

Поля структуры располагаются в памяти в том порядке, в котором они объявлены

При объявлении структур, их разрешается вкладывать одну в другую.

```
1  struct point
2  {
3    double x;
4    double y;
5    double z;
6  };
7  struct rectangle
8  {
9    double width;     // ширина
10   double height;     // высота
11   struct point leftTop;     // координата ↩
       угла
12 };
```

Инициализация полей структуры

```
struct <ИмяСтруктуры> <ИмяПеременной>=
{<ЗначениеЭлемента1>, <ЗначениеЭлемента_2>, . . . ,

struct date bd={8, 6, 1978};
```

Порядок и тип аргументов, должен совпадать с порядком и типом полей, описанных в шаблоне структуры

Имя элемента структуры является составным. Для обращения к элементу структуры нужно указать имя структуры и имя самого элемента. Они разделяются точкой:

```
ИмяПеременной.ИмяЭлементаСтруктуры
```

```
printf("%d %d %d",bd.day, bd.month, bd.year);
```

Для вложенных структур.

```
1  #include <stdio.h>
2  struct point
3  {
4    double x;
5    double y;
6    double z;
7  };
8  struct rectangle
9  {
10   double width;
11   double height;
12   struct point leftTop;
13 };
```

```
1  int main()
2  { struct rectangle p;
3    printf("Введите ширину : ");
4    scanf("%lf", &p.width);
5    printf("Введите высоту: ");
6    scanf("%lf", &p.height);
7    printf("Введите координату угла \n ");
8    scanf("%d", &p.leftTop.x);
9    scanf("%d", &p.leftTop.y);
10   scanf("%d", &p.leftTop.z);
11   return 0;
12 }
```

Структуры

Имя структурной переменной может быть указано при объявлении структуры. В этом случае оно размещается после закрывающей фигурной скобки }. Область видимости такой структурной переменной будет определяться местом описания структуры.

```
1 struct complex_type  // имя структуры
2 {
3   double real;
4   double imag;
5 } number;     // имя структурной переменной
```

Поля приведенной структурной переменной: number.real, number.imag

Массивы в С

Массив - структура данных, содержащая набор проиндексированных элементов. В языке С массивы являются типизированными, то есть могут содержать элементы только одного типа.

```
<тип> <имя_массива>[<размер>];
```

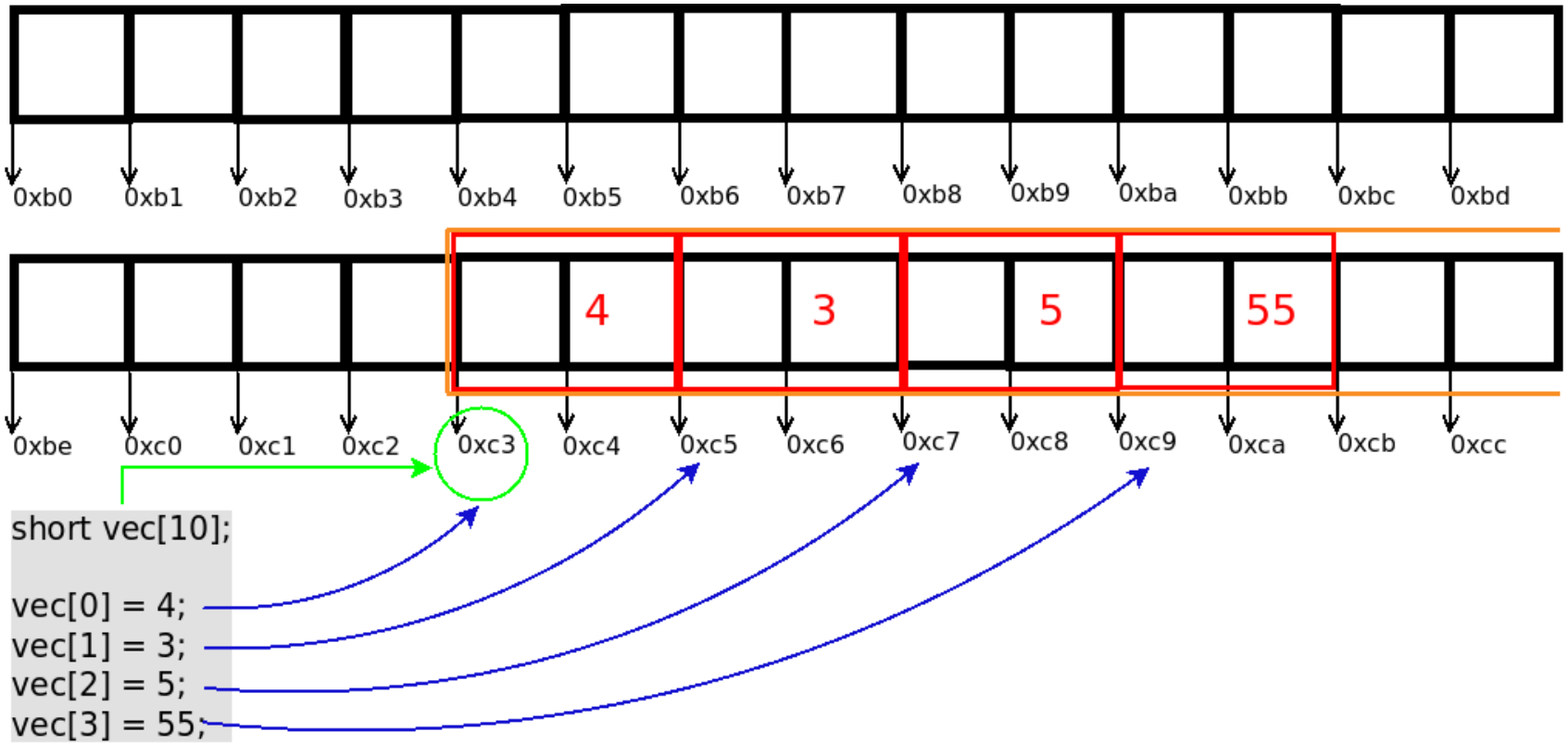
В С все элементы одного массива располагаются в памяти последовательно.

Статический массив

Индексация элементов в массиве **начинается с нуля**.

```
1 short vec[10];
2
3 // Задаём значение первого элемента
4 vec[0] = 4;
5 vec[3] = 55;
6
7 printf("Первый элемент равен %d", vec[0]);
```

Точка зрения памяти



Массивы в С

Инициализация массива - задание начальных значений каждому элементу

```
1  int vec[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };
2
3  for (unsigned i = 0; i < 8; ++i) {
4    printf("%u ", vec[i]);
5  }
6  printf("\n");
7
8  // Незаданные элементы будут равны 0.0
9  double real_arr[5] = { 3.4, 5.5, 77.11 };
10
11 // При явной инициализации размер массива ↩
     можно пропускать
12 int another_vec[] = { 1, 2, 3, 4 };
```

Многомерные массивы

```
1  int matrix1[10][10];
2
3  for (int i = 0; i < 10; ++i) {
4    for (int j = 0; j < 10; ++j) {
5      matrix[i][j] = i + j;
6    }
7  }
8
9  // Инициализация
10 int matrix2[3][3] = { {1, 2, 3}, {4, 5, 6}, ↩
     {7, 8, 9} };
```

Многомерные массивы: в числе размерностей никто не ограничен. Гарантируется работа до 31 уровня вложенности.

```
1 int monstr[3][4][5][3][4][5];
2 monstr[0][0][0][0][0][0] = 5;
```

Передача массивов в функции.

```
1  void print_array(int arr[], size_t count)
2  {
3    std::cout << "\nПереданный массив:\n";
4    for (size_t i = 0; i < count; ++i) {
5      printf("%d ", arr[i]);
6    }
7    printf("\n");
8  }
9
10 ...
11
12 int vec[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };
13 print_array(vec, 8);
```

Никакой проверки размерности массива не происходит.

```
1  void print_array(int arr[55], size_t count)
2  {
3    printf("\nПереданный массив:\n");
4    for (size_t i = 0; i < count; ++i) {
5      printf("%d ", arr[i]);
6    }
7    printf("\n");
8  }
9
10 ...
11
12 int vec[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };
13 print_array(vec, 8);
```

Массивы в C

Многомерные массивы: нужно указать все размерности, кроме первой.

```
1  const size_t COLS = 8;
2
3  void print_2D_array(int matr[][COLS], size_t↩
       rows_count)
4  {
5    printf("\nПереданный двумерный массив:\n")↩
       ;
6    for (size_t i = 0; i < rows_count; ++i) {
7      for (size_t j = 0; j < COLS; ++j) {
8        printf("%d ", matr[i][j]);
9      }
10     printf("\n");
11   }
12   printf("\n");
13 }
```

Массивы в C

В современном стандарте C (**C11**) определены **массивы переменной длины** (VLA - variable length array)

```
1 size_t my_size = 15;
2
3 int arr[my_size];
4
5 for (size_t i = 0; i < my_size; ++i) {
6   arr[i] = 15 - i;
7 }
```

Другой пример на **массивы переменной длины**

```
1  size_t rows, cols;
2
3  printf("Введите число строки и"
4         " столцов матрицы: ");
5  scanf("%lu %lu", &rows, &cols);
6
7  // Оперделяем двумерный массив
8  // заданных размеров
9  double real_matrix[rows][cols];
10
11 // Используем по своему усмотрению
12 for (size_t i = 0; i < rows; ++i) {
13   for (size_t j = 0; j < cols; ++j) {
14     real_matrix[i][j] = 35.678 /(1 + i + j);
15   }
16 }
```

Ограничения на **массивы переменной длины**

- Массивы переменной длины не могут быть глобальными (должны использоваться только внутри функций)
- Массив переменной длины невозможно проинициализировать начальными значениями при объявлении

Общее ограничение массивов

Массивы в С сравниваются только **поэлементно**. Сравнивать переменные массивов (такие как **arr, real_matrix** в примерах выше) - бесполезно, при этом сравниваются только их адреса